

Exp No: 8	MODEL EVALUATION AND IMPROVEMENT: HYPERPARAMETER TUNING WITH GRID SEARCH AND CROSS-VALIDATION
------------------	--

```
# EXPERIMENT 8
# Hyperparameter Tuning with Grid Search and Cross-Validation

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_iris
from sklearn.model_selection import (
    train_test_split,
    KFold,
    cross_val_score,
    GridSearchCV
)
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix
)

# Load Iris dataset
iris = load_iris()

X = iris.data
y = iris.target
```

```

feature_names = iris.feature_names
target_names = iris.target_names

print("Dataset Features Shape:", X.shape)
print("Dataset Labels Shape:", y.shape)
print("Feature Names:", feature_names)
print("Target Names:", target_names)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42,
    stratify=y
)

print("\nTraining Set Size:", X_train.shape[0])
print("Testing Set Size:", X_test.shape[0])

# Standardize features
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("\nFeatures standardized successfully.")

# Define SVM model
svm = SVC()

# Define hyperparameter grid
param_grid = {
    "C": [0.1, 1, 10, 100],
    "gamma": [1, 0.1, 0.01, 0.001],
    "kernel": ["rbf", "linear"]
}

print("\nHyperparameter Grid:")
print(param_grid)

# Grid Search with 5-fold Cross-Validation
grid_search = GridSearchCV(
    estimator=svm,
    param_grid=param_grid,

```

```

        cv=5,
        scoring="accuracy",
        n_jobs=-1
    )

    print("\nStarting Grid Search...")

    grid_search.fit(X_train_scaled, y_train)

    print("Grid Search completed successfully.")

    # Best parameters
    print("\nBest Hyperparameters:")
    print(grid_search.best_params_)

    print("\nBest Cross-Validation Accuracy:")
    print(round(grid_search.best_score_, 4))

    # Best model
    best_model = grid_search.best_estimator_

    # Prediction
    y_pred = best_model.predict(X_test_scaled)

    # Test accuracy
    test_accuracy = accuracy_score(y_test, y_pred)

    print("\nTest Set Accuracy:")
    print(round(test_accuracy, 4))

    # Classification report
    print("\nClassification Report:")
    print(
        classification_report(
            y_test,
            y_pred,
            target_names=target_names
        )
    )

    # Confusion matrix
    cm = confusion_matrix(y_test, y_pred)

    print("\nConfusion Matrix:")
    print(cm)

```

```

plt.figure(figsize=(7, 5))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=target_names,
    yticklabels=target_names
)

plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.title("Confusion Matrix - Tuned SVM")

plt.show()

# Display top Grid Search results
results_df = pd.DataFrame(grid_search.cv_results_)

top_results = results_df[
    [
        "param_C",
        "param_gamma",
        "param_kernel",
        "mean_test_score",
        "rank_test_score"
    ]
].sort_values("rank_test_score").head(10)

print("\nTop Grid Search Results:")
print(top_results.to_string(index=False))

# K-Fold Cross-Validation
print("\n" + "=" * 50)
print("K-FOLD CROSS-VALIDATION")
print("=" * 50)

model_cv = SVC(
    C=grid_search.best_params_["C"],
    gamma=grid_search.best_params_["gamma"],
    kernel=grid_search.best_params_["kernel"]
)

```

```

k_folds = 5

kf = KFold(
    n_splits=k_folds,
    shuffle=True,
    random_state=42
)

cv_scores = cross_val_score(
    model_cv,
    X_train_scaled,
    y_train,
    cv=kf,
    scoring="accuracy"
)

print("\nCross-Validation Scores:")
print(cv_scores)

print("\nMean Cross-Validation Accuracy:")
print(round(np.mean(cv_scores), 4))

print("\nStandard Deviation:")
print(round(np.std(cv_scores), 4))

# Plot CV scores
plt.figure(figsize=(8, 5))

plt.bar(
    range(1, k_folds + 1),
    cv_scores
)

plt.axhline(
    y=np.mean(cv_scores),
    linestyle="--",
    label="Mean Accuracy"
)

plt.xlabel("Fold Number")
plt.ylabel("Accuracy")
plt.title("5-Fold Cross-Validation Accuracy")

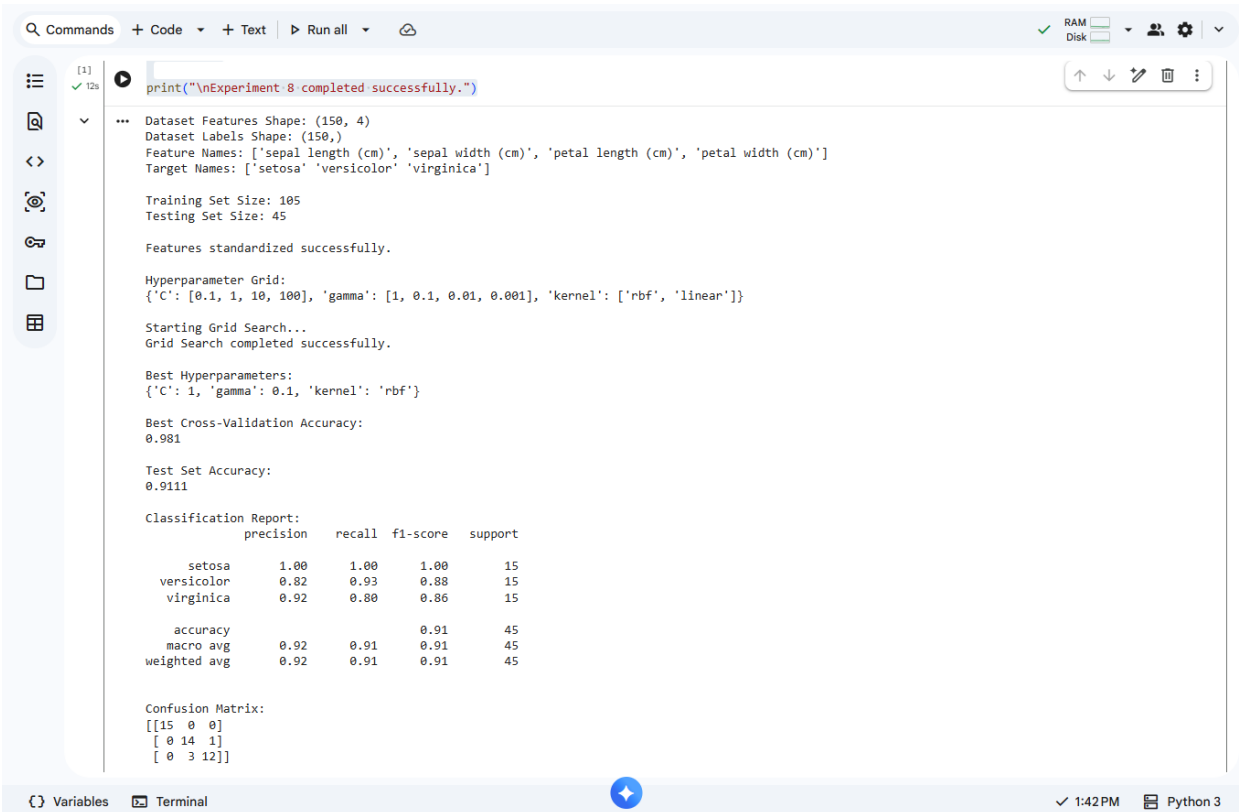
plt.legend()
plt.grid(axis="y")

```

```
plt.show()
```

```
print("\nExperiment 8 completed successfully.")
```

OUTPUT



The screenshot shows a Jupyter Notebook interface with a light blue header bar. The top bar contains a search icon, the text "Commands", and several icons for code, text, and running. On the right side of the header bar, there are icons for RAM and Disk usage, a user profile icon, and a settings gear icon. The main area of the notebook displays the output of a Python script. The output starts with a print statement: `print("\nExperiment 8 completed successfully.")`. This is followed by a series of text outputs: "Dataset Features Shape: (150, 4)", "Dataset Labels Shape: (150,)", "Feature Names: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']", "Target Names: ['setosa' 'versicolor' 'virginica']", "Training Set Size: 105", "Testing Set Size: 45", "Features standardized successfully.", "Hyperparameter Grid: {'C': [0.1, 1, 10, 100], 'gamma': [1, 0.1, 0.01, 0.001], 'kernel': ['rbf', 'linear']}", "Starting Grid Search...", "Grid Search completed successfully.", "Best Hyperparameters: {'C': 1, 'gamma': 0.1, 'kernel': 'rbf'}", "Best Cross-Validation Accuracy: 0.981", "Test Set Accuracy: 0.9111". Then, a "Classification Report:" is shown as a table with columns for precision, recall, f1-score, and support. The rows include 'setosa', 'versicolor', 'virginica', 'accuracy', 'macro avg', and 'weighted avg'. Finally, a "Confusion Matrix:" is displayed as a 3x3 array.

```
[1]
✓ 12s
print("\nExperiment 8 completed successfully.")

*** Dataset Features Shape: (150, 4)
Dataset Labels Shape: (150,)
Feature Names: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
Target Names: ['setosa' 'versicolor' 'virginica']

Training Set Size: 105
Testing Set Size: 45

Features standardized successfully.

Hyperparameter Grid:
{'C': [0.1, 1, 10, 100], 'gamma': [1, 0.1, 0.01, 0.001], 'kernel': ['rbf', 'linear']}

Starting Grid Search...
Grid Search completed successfully.

Best Hyperparameters:
{'C': 1, 'gamma': 0.1, 'kernel': 'rbf'}

Best Cross-Validation Accuracy:
0.981

Test Set Accuracy:
0.9111

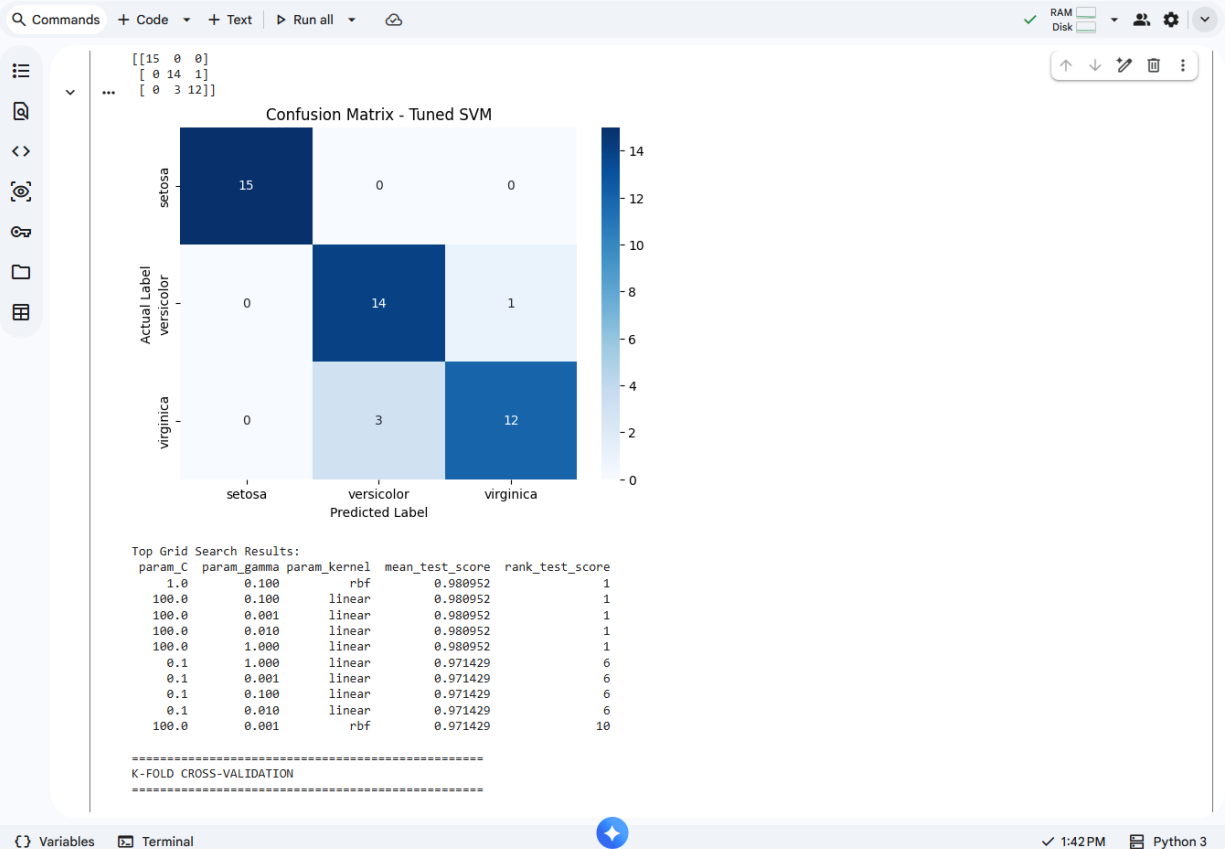
Classification Report:
      precision    recall  f1-score   support

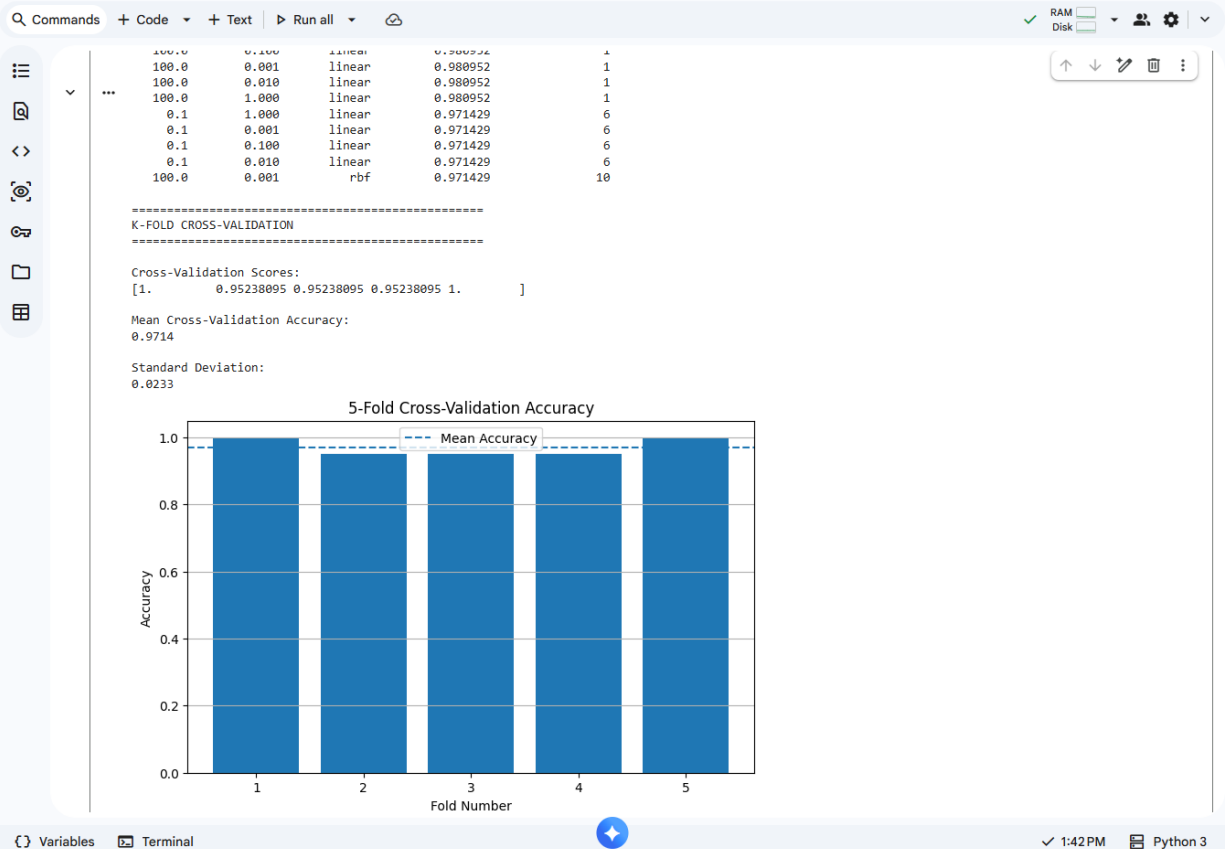
   setosa       1.00      1.00      1.00        15
  versicolor    0.82      0.93      0.88        15
   virginica    0.92      0.80      0.86        15

 accuracy       0.92      0.91      0.91        45
  macro avg       0.92      0.91      0.91        45
 weighted avg       0.92      0.91      0.91        45

Confusion Matrix:
[[15  0  0]
 [ 0 14  1]
 [ 0  3 12]]
```

Variables Terminal 1:42 PM Python 3





CODE 2:

EXPERIMENT 8

Hyperparameter Tuning using Grid Search and Cross-Validation

Dataset: Breast Cancer Wisconsin

Model: Random Forest

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split, GridSearchCV, cross_val_score

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

Load dataset

data = load_breast_cancer()

X = data.data

y = data.target

print("Dataset Name: Breast Cancer Wisconsin")

print("Number of Samples:", X.shape[0])

print("Number of Features:", X.shape[1])

print("Target Classes:", data.target_names)

Convert to DataFrame

```
df = pd.DataFrame(  
    X,  
    columns=data.feature_names  
)  
  
df["Target"] = y  
  
print("\nFirst 5 Rows:")  
print(df.head())  
  
# Check missing values  
print("\nMissing Values:")  
print(df.isnull().sum().sum())  
  
# Split dataset  
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.2,  
    random_state=42,  
    stratify=y  
)  
  
print("\nTraining Samples:", len(X_train))  
print("Testing Samples:", len(X_test))  
  
# Create Random Forest model  
rf = RandomForestClassifier(  
    random_state=42
```

```
)
```

```
# Define hyperparameter grid
```

```
param_grid = {  
    "n_estimators": [50, 100, 150],  
    "max_depth": [None, 5, 10],  
    "min_samples_split": [2, 5]  
}
```

```
print("\nStarting Grid Search...")
```

```
# Grid Search with 5-fold cross-validation
```

```
grid_search = GridSearchCV(  
    estimator=rf,  
    param_grid=param_grid,  
    cv=5,  
    scoring="accuracy",  
    n_jobs=-1  
)
```

```
grid_search.fit(X_train, y_train)
```

```
print("Grid Search Completed.")
```

```
# Display best parameters
```

```
print("\nBest Parameters:")  
print(grid_search.best_params_)
```

```
# Best cross-validation score
```

```
print("\nBest Cross-Validation Accuracy:")
print(round(grid_search.best_score_, 4))

# Get best model
best_model = grid_search.best_estimator_

# Predict test data
y_pred = best_model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("\nTest Accuracy:")
print(round(accuracy, 4))

# Classification report
print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred,
        target_names=data.target_names
    )
)

# Confusion matrix
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
```

```
print(cm)
```

```
# Plot confusion matrix manually
```

```
plt.figure(figsize=(6, 5))
```

```
plt.imshow(cm)
```

```
plt.title("Confusion Matrix - Tuned Random Forest")
```

```
plt.xlabel("Predicted Class")
```

```
plt.ylabel("Actual Class")
```

```
plt.xticks(  
    [0, 1],  
    data.target_names  
)
```

```
plt.yticks(  
    [0, 1],  
    data.target_names  
)
```

```
for i in range(2):
```

```
    for j in range(2):
```

```
        plt.text(  
            j,  
            i,  
            cm[i, j],  
            ha="center",  
            va="center"
```

```
)
```

```
plt.colorbar()
```

```
plt.show()
```

```
# Cross-validation of best model
```

```
print("\nPerforming 5-Fold Cross-Validation...")
```

```
cv_scores = cross_val_score(
```

```
    best_model,
```

```
    X_train,
```

```
    y_train,
```

```
    cv=5,
```

```
    scoring="accuracy"
```

```
)
```

```
print("\nCross-Validation Scores:")
```

```
print(cv_scores)
```

```
print("\nMean Cross-Validation Accuracy:")
```

```
print(round(np.mean(cv_scores), 4))
```

```
print("\nStandard Deviation:")
```

```
print(round(np.std(cv_scores), 4))
```

```
# Plot cross-validation scores
```

```
plt.figure(figsize=(8, 5))
```

```
plt.bar(
```

```
    range(1, 6),  
    cv_scores  
)  
  
plt.axhline(  
    np.mean(cv_scores),  
    linestyle="--",  
    label="Mean Accuracy"  
)  
  
plt.xlabel("Fold Number")  
plt.ylabel("Accuracy")  
plt.title("5-Fold Cross-Validation")  
  
plt.xticks(range(1, 6))  
  
plt.legend()  
plt.grid(axis="y")  
  
plt.show()  
  
print("\nExperiment 8 completed successfully.")
```

OUTPUT:

Dataset Name: Breast Cancer Wisconsin
Number of Samples: 569
Number of Features: 30
Target Classes: ['malignant' 'benign']

First 5 Rows:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	\
0	17.99	10.38	122.80	1001.0	0.11840	
1	20.57	17.77	132.90	1326.0	0.08474	
2	19.69	21.25	130.00	1203.0	0.10960	
3	11.42	20.38	77.58	386.1	0.14250	
4	20.29	14.34	135.10	1297.0	0.10030	

	mean compactness	mean concavity	mean concave points	mean symmetry	\
0	0.27760	0.3001	0.14710	0.2419	
1	0.07864	0.0869	0.07017	0.1812	
2	0.15990	0.1974	0.12790	0.2069	
3	0.28390	0.2414	0.10520	0.2597	
4	0.13280	0.1980	0.10430	0.1809	

	mean fractal dimension	...	worst texture	worst perimeter	worst area	\
0	0.07871	...	17.33	184.60	2019.0	
1	0.05667	...	23.41	158.80	1956.0	
2	0.05999	...	25.53	152.50	1709.0	
3	0.09744	...	26.50	98.87	567.7	
4	0.05883	...	16.67	152.20	1575.0	

	worst smoothness	worst compactness	worst concavity	worst concave points	\
0	0.1622	0.6656	0.7119	0.2654	
1	0.1238	0.1866	0.2416	0.1860	
2	0.1444	0.4245	0.4504	0.2430	
3	0.2098	0.8663	0.6869	0.2575	
4	0.1374	0.2050	0.4000	0.1625	

	worst symmetry	worst fractal dimension	Target
0	0.4601	0.11890	0
1	0.2750	0.08902	0
2	0.3613	0.08758	0
3	0.6638	0.17300	0
4	0.2364	0.07678	0

[5 rows x 31 columns]

Missing Values:
0

Training Samples: 455
Testing Samples: 114

Starting Grid Search...
Grid Search Completed.

Best Parameters:
{ 'max_depth': None, 'min_samples_split': 2, 'n_estimators': 150 }

Best Cross-Validation Accuracy:
0.9604

Test Accuracy:
0.9561

Classification Report:

	precision	recall	f1-score	support
malignant	0.95	0.93	0.94	42
benign	0.96	0.97	0.97	72
accuracy			0.96	114
macro avg	0.96	0.95	0.95	114
weighted avg	0.96	0.96	0.96	114

Confusion Matrix:

```
[[39  3]
 [ 2 70]]
```

Performing 5-Fold Cross-Validation...

Cross-Validation Scores:

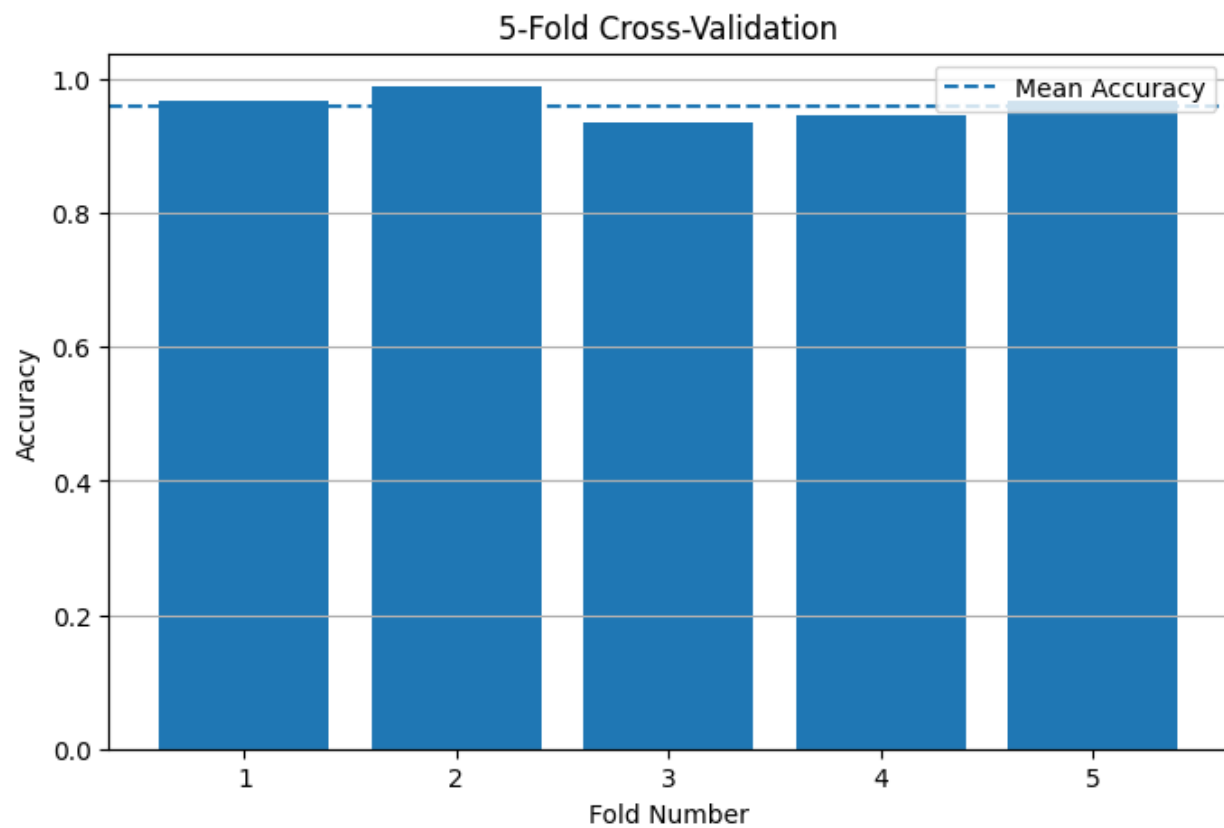
```
[0.96703297 0.98901099 0.93406593 0.94505495 0.96703297]
```

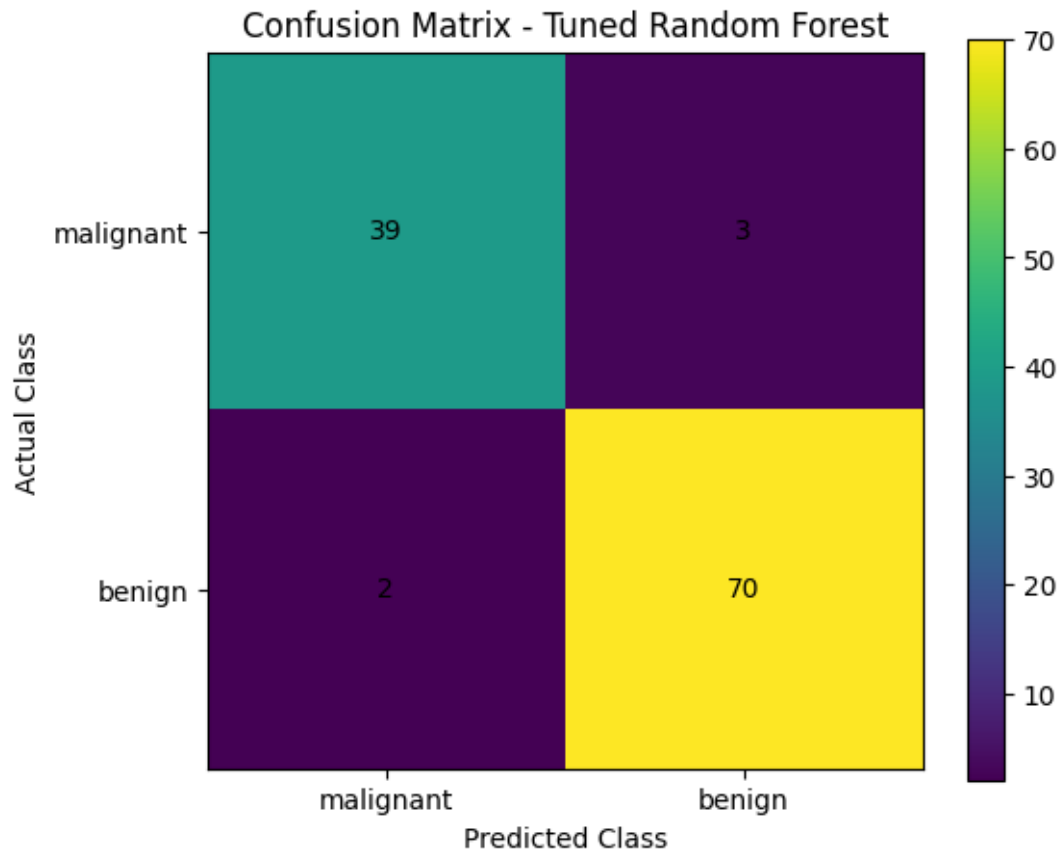
Mean Cross-Validation Accuracy:

0.9604

Standard Deviation:

0.0192





RESULT:

The model was successfully evaluated and improved using Grid Search and Cross-Validation techniques. Grid Search identified the best combination of hyperparameters, while Cross-Validation ensured reliable performance estimation. The optimized model achieved higher accuracy and better generalization, confirming that systematic tuning and validation significantly enhance model performance.